CareerConnect – AI Usage Log

Team Member: Vlad Shkuratov | Tool: Claude Code (Claude Sonnet 5) | Sessions: September 27 and October 25–26, 2026

Prompts below were shortened and cleaned up for readability. Their meaning was not changed. Routine questions about using Git (merging, publishing a branch) are not logged because AI output did not influence any project deliverable in those cases.

Sprint 1 Session (September 27, 2026)

Task 1.1: Backend Fails to Start Locally

Purpose of AI Use
Debugging. The FastAPI backend would not start with the command from the team README.

Prompt / Response

> ***Prompt (lightly edited for clarity):*** *Why does uv run fastapi dev backend/main.py not run? (Followed by a screenshot of the next error.)*

Response: The AI ran the command, found the path was wrong (main.py lives in src/backend/), then compared .env with .env.example and found a typo in the variable name.

AI-Suggested Content
Use the correct path *src/backend/main.py*, and rename JWT_SECRET_KET to JWT_SECRET_KEY in the local .env file.

Validation
Restarted the backend after the change and confirmed it started and served requests.

Decision
Accepted.

Reflection
Both problems were small, but the second one would have been hard to spot by eye because the error did not name the variable. Comparing against .env.example was the useful step.

Responsible Person
Vlad Shkuratov

Task 1.2: Signup and Login UI – US-01.01 (Issue #36)

Purpose of AI Use
Code generation and learning. This was my assigned task, and I asked for plain-language explanations because I am new to React.

Prompt / Response

> ***Prompt (lightly edited for clarity):*** *Build the signup and login pages for issue #36 in the existing React 19 + Vite + Tailwind setup, using plain JavaScript and no new dependencies. Call POST /api/auth/register and /api/auth/login through a Vite dev proxy, handle every error response (400, 401, 403 lockout, 422 with a list-shaped detail), show the password rules in the UI, store the access token on login, and add a logout button. Split it into small components with the fetch calls in one API file. Do not touch .env, README.md or the database, and do not commit or push.*

Response: Before writing UI code, the AI tested the backend and found that POST /api/auth/register was broken: the route decorator was attached to the wrong function and an *import re* was missing. It asked whether to fix it (I said yes), then built the UI.

AI-Suggested Content
A Vite proxy for /api; api/auth.js with one error-parsing helper for both error shapes; passwordRules.js mirroring the backend rules; components SignupForm, LoginForm, PasswordRequirements (live checklist) and LoggedInScreen; view switching with one state variable in App.jsx; removal of the Vite demo styles. Separately, a two-line backend fix for the register route (issue #45).

Validation
Tested register (success, weak password, duplicate email) and login (success, wrong password) with curl against the real backend, before and after the fix. Confirmed the proxy forwarded requests, then ran a full browser pass: signup, live password checklist, redirect to login, login, logout, and the wrong-password error. No console errors. I also reviewed the explanations of each file before committing.

Decision
Accepted. The backend fix was applied only after I approved it, since it was outside my issue.

Reflection
Useful for learning how state, props and callbacks connect in React. The more important result was catching the register bug, because the UI could not have been tested end to end without it.

Responsible Person
Vlad Shkuratov

Task 1.3: Teammate Login Crash from an Outdated Local Database

Purpose of AI Use
Debugging. A teammate (Ron) got a long error on every login attempt saying the column user.failed_attempts did not exist.

Prompt / Response

> ***Prompt (lightly edited for clarity):*** *Here is the error my teammate gets when logging in. Find the issue and fix it.*

Response: The AI identified it as schema drift: the lockout feature added two columns to the User model, but *create_all()* only creates missing tables and never adds columns to existing ones. Anyone whose local database file was older than that change would crash.

AI-Suggested Content

A helper in database.py that runs at startup, compares each table's real columns with the model, and adds any missing ones with ALTER TABLE (using the model default for NOT NULL columns). It does nothing when the database is already up to date.

Validation
Built a copy of the database with the old schema, ran the startup code on it, and checked with PRAGMA table_info that both columns were added correctly. Then ran register, login and wrong-password login against it. During cleanup the AI's attempt to restore the temporary database path did not apply, which it caught and corrected before finishing. A final login against my real database confirmed it still worked.

Decision
Accepted. Teammates only needed to pull and restart.

Reflection
This would have kept happening every time a model changed, because the project has no migration tool. The fix is a workaround, so a real migration tool such as Alembic is still worth discussing with the team.

Responsible Person
Vlad Shkuratov

Sprint 2 Session (October 25–26, 2026)

Task 2.1: Sprint 2 Requirements Gap Audit

Purpose of AI Use
Requirements analysis and review. Several teammates had merged work separately, and it was unclear what was finished compared to the Sprint 2 requirements.

Prompt / Response

> ***Prompt (lightly edited for clarity):*** *Here is the Sprint 2 description. Check the code on main and tell me if the team is missing anything. Follow-ups: focus on the code itself; are US-04, US-06 and US-08 meant for Sprint 2 or a later sprint; what does the "status tracking badges" task assigned to me actually mean?*

Response: The AI read the Sprint 2 document and inspected the repository: backend routers and models, frontend components, tests, CI workflow, and the documentation folders.

AI-Suggested Content
Profile and resume management was complete. Job search and application submission had working backends but no frontend. The recruiter dashboard was missing an endpoint to list received applications (allowed to be partial this sprint). There were no tests for the application endpoints, and MeetingMinutes and most AI_Log folders only had placeholders. It confirmed from the issue milestones that US-04, US-06 and US-08 are Sprint 2 scope, and explained that the badges task meant a new color-coded status component.

Validation
Ran the full pytest suite and checked findings against the actual source files and the requirement PDFs rather than file names alone. GitHub issues and the project board could not be checked from the terminal, so I checked those myself.

Decision
Accepted. It set the priorities for the rest of the sprint.

Reflection
A quick look at the app would not have shown that two required features had no UI. The audit also showed that the team's FIXES.md was partly out of date.

Responsible Person
Vlad Shkuratov

Task 2.2: Application Status Tracking Badges – US-08.02 (Issue #97)

Purpose of AI Use
Code generation for my assigned Sprint 2 sub-issue.

Prompt / Response

> ***Prompt (lightly edited for clarity):*** *I made a new branch. Build the status tracking badges.*

Response: The AI read the existing ProfileSection, ResumeSection and API files to match the team's conventions, then built the feature and seeded test data to check it.

AI-Suggested Content
ApplicationStatusBadge.jsx (one color per status: Applied blue, Under Review yellow, Interview purple, Offered green, Rejected red); ApplicationsSection.jsx (list of the user's applications with a status filter and a withdraw action); api/applications.js; and wiring into the dashboard.

Validation
Seeded jobs, created a test user with a resume, applied to three jobs, and used the recruiter account to move them to different statuses. A browser pass confirmed distinct badge colors, a working filter, and withdraw with a success message, with no console errors. Checked against both acceptance criteria on issue #97. I also clicked through it myself on localhost.

Decision
Accepted.

Reflection
Matching the team's existing styles made the section look consistent with the rest of the dashboard without extra design work.

Responsible Person
Vlad Shkuratov

Task 2.3: Job Search Fixes and Application Submission – US-04.02 (#82), US-06.02 (#94)

Purpose of AI Use
Code review, debugging and code generation. After the job search page was merged into main, I asked for a full re-check of Sprint 1 and 2 requirements on the running app.

Prompt / Response

> ***Prompt (lightly edited for clarity):*** *Run everything against the Sprint 1 and Sprint 2 requirements and flag any feature that is missing or not working. Then: fix all of them and run everything again to make sure it works.*

Response: The AI ran the tests, build and linter, read the merged job search code, and found that the page was only reachable through a dev-only URL, had no category filter (required by US-04), had a Reset button that did not clear results, had a Log out button that did nothing, and had no Apply button anywhere in the app.

AI-Suggested Content
A new api/jobs.js client; a category filter and Reset fix in JobSearchPageNavbar.jsx; a fuller job detail panel with an Apply button and success and error messages; a Browse Jobs link in the main navbar and a Dashboard link back; the Log out props wired through; and removal of the lint warnings.

Validation
Lint went from 5 warnings to 0, the production build passed, and the backend suite stayed at 35/35. The apply flow was verified against the live backend (201 on first apply, 409 on a duplicate) and the new application appeared in My Applications with its badge. No browser tool was available for this last pass, so it relied on API calls and source review, and I checked the UI by hand afterwards.

Decision
Accepted. These changes overlap two teammates' open issues, so this was documented in the commit message and PR description so contribution tracking stays accurate.

Reflection
The page was merged and passed CI, but core acceptance criteria were never built. That only showed up by using the feature end to end, which is a gap in our review process rather than in any single PR.

Responsible Person
Vlad Shkuratov

Task 2.4: Commit Messages and Pull Request Descriptions

Purpose of AI Use
Documentation writing.

Prompt / Response

> ***Prompt (lightly edited for clarity):*** *Write the commit message, then a PR description that follows the team's pull request template and that I can paste straight into GitHub.*

Response: The AI read the staged diff, the recent commit history for style, and .github/pull_request_template.md.

AI-Suggested Content
Commit messages in the repository's existing style with issue numbers, and PR descriptions filled into the template (Summary with the closed and related issues, Changes Made, Checklist).

Validation
Compared each description with the actual diff against main to make sure it listed only what changed.

Decision
Accepted.

Reflection
Saves time, but the description is only as accurate as the diff check behind it, so that step is worth keeping.

Responsible Person
Vlad Shkuratov